

# 实验六：协议实现-在网计算

24 : 09 : 27 : 14  
天 时 分 秒

## 实验目的

1. 综合运用前几次实验的能力：基于 libpcap 的原始帧收发、可靠传输、路由转发；
2. 自行设计并实现一个极简传输协议（在 IP 之上，完全掌控分组边界）；
3. 理解在网计算（In-Network Computing, INC）技术，实现集合通信原语 Shift 与 AllReduce。

## 实验背景：此前基于TCP的构造不成功，原因分析

在网计算的核心思想是：把本应在主机端完成的计算（这里是 AllReduce 的“求和”）下沉到交换机/路由器里完成。交换机里有一组“聚合单元”（aggregator），每个单元负责把**同一个序号位置**上来自各个成员的载荷累加起来。这要求：

每一个数据分组（packet）的载荷长度，必须与交换机里聚合单元的长度严格一致，且分组与聚合单元一一对应。

这就是旧版本实验（在归档的实验中）的根本问题所在：旧版本用 **TCP socket** 收发。TCP 提供的是**字节流**抽象，应用层调用 `send()` 交出的数据，会被 TCP 任意地拆分/合并成报文段（受 Nagle 算法、GSO/TSO 分段卸载、MSS 协商、拥塞窗口等影响），**应用层无法控制实际分组的载荷长度**。于是交换机收到的报文段载荷时大时小，无法稳定地映射到聚合单元，聚合结果随之损坏。旧代码不得不靠“关闭 Nagle、关闭网卡 offload、关闭 TCP 时间戳、设置 TCP\_MAXSEG、检测到非 MSS 载荷就直接 `exit(1)`”等一系列脆弱的补丁勉强维持，本质上是在和 TCP 的字节流语义对抗。

**本实验的做法**：抛弃 socket，改用 **libpcap** 直接收发二层原始帧（与实验四相同的 IO 模式），保留 IP 首部，但在 IP 之上**自己实现一个极简的传输协议 MTP**。这样应用层对每一个分组的边界和载荷长度有完全的控制权，聚合单元与分组严格一一对应，在网计算才得以正确实现。

## 实验准备

1. 复习**实验四（路由协议）**：`tutorial` 中的 libpcap 收发模式（`pcap_open_live`、抓包线程、`pcap_inject`）。本实验的收发接口完全建立在这个模式之上。

2. 复习**实验五（传输协议）**：滑动窗口、超时重传、独立 ACK、选择重传的思想。本实验的传输逻辑是它的简化版（固定窗口、无拥塞控制）。
3. 阅读参考文献：
  - a. **NetReduce [ASPLOS 2023]**：用可编程网卡/交换机做 AllReduce，重点理解“聚合器 + 位图 + 基于序列号寻址 + 丢包靠主机重传恢复”的设计。该工作获 2024 年中国算力大会最佳学术论文奖。
  - b. **EPIC (Mode-II) [Arxiv 2026]**：进一步理解聚合器在消息大于聚合器数组时如何循环复用 (recycling)。

## Clab云环境配置

参考“实验准备”中更新的部分，在clab上启动云主机：[链接](#)

关于Clab中的环境配置の説明：

1. Clab中的Ubuntu镜像中的apt源是配置好的，可以直接使用 `apt` 命令配置和更新环境
2. 可以使用apt命令安装docker.io
3. Clab云主机默认不连接互联网；另外，docker官方镜像网站也有封禁，我们需要在docker中加载 ubuntu 22.04容器镜像

我们采用下面方法：

1. 在云主机安装docker
2. 从“实验课”文件夹中下载“ubuntu2204.tar.gz”镜像文件，上传到云主机上
3. 运行命令 `docker load -i ubuntu2204.tar.gz` 将镜像文件加载为docker image
4. 注意，官方镜像在clab环境下运行仍然有两个问题
  - a. apt源是互联网的源，仍然不能访问
  - b. 镜像没有证书，访问源时不能通过验证
5. 我们将clab环境的sources.list文件和对应的权限文件准备好，构建Dockerfile时添加两行，将这两个文件复制到容器内部（目前Dockerfile已经包含）

代码块

```
1  # Base images 基础镜像
2  FROM ubuntu:22.04
3
4  # RUN timedatectl set-timezone Asia/Shanghai'
5  COPY sources.list /etc/apt/sources.list
6  COPY 99clab-noverify /etc/apt/apt.conf.d/99clab-noverify
7  RUN apt update
```

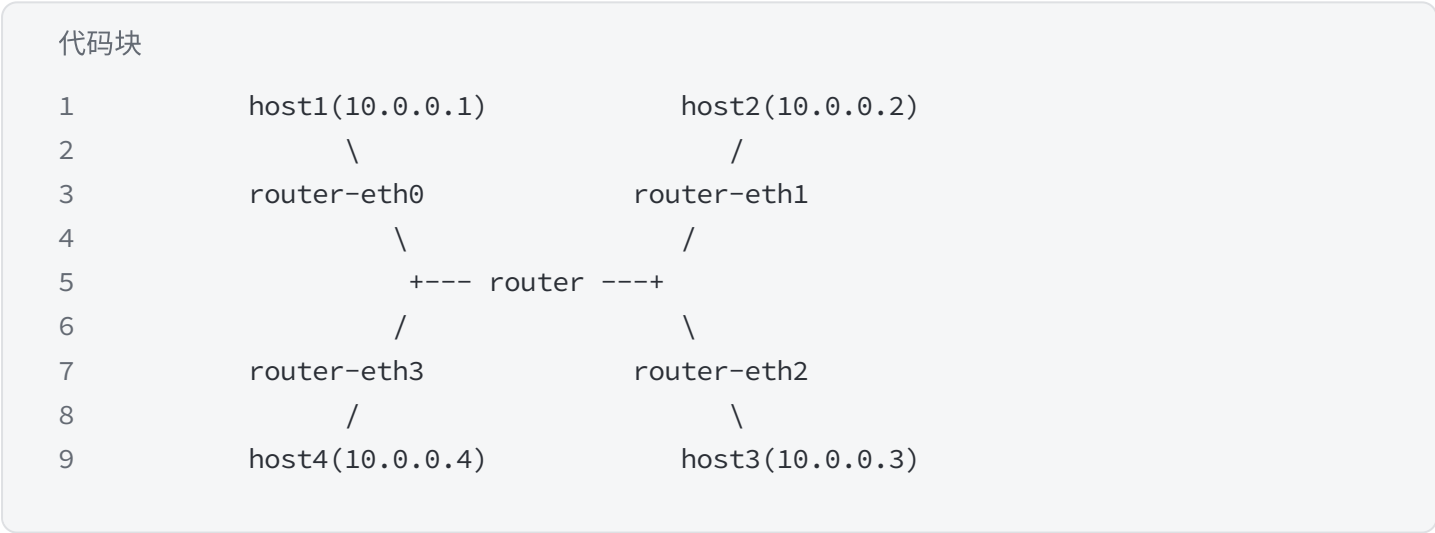
完成以上步骤，实验的软件环境和开发环境就配置好了。参考下面的“开发与测试”中的 `make setup_env` 命令，观察环境配置的具体步骤。

## 服务模型

1. 一个并程序里有若干运行在主机端的进程，每个进程有唯一编号 **Rank**，从 0 依次编号；
2. 若干进程组成一个**通信组（group）**；本实验让 4 个主机组成 1 个通信组；
3. 组内成员可进行多种集合通信原语。本实验实现两种：
  - a. **Shift**：每个 rank 把自己的数据发给环上的后继 rank（环形移位）；
  - b. **AllReduce**：组内所有 rank 的数据按元素求和，结果分发回每个 rank。

## 部署模型

### 拓扑（4 hosts + 1 router 的星形）



- 4 个 host 各用一条链路连到 router 的一个端口；
- **逻辑上**，4 个 host 按 Rank 组成一个**环**：0 → 1 → 2 → 3 → 0；
- 每个 host 与环上的**前驱**、**后继**建立连接：向后继发数据，从前驱收数据。

### 南向接口：libpcap 收发原始帧

主机与路由器都不依赖内核协议栈，全部通过 libpcap 抓包/注入。因此拓扑脚本无需配置内核路由、ARP，也无需关闭 offload（我们不走内核 IP 层）。

## 协议设计

### 帧格式

- **以太网首部**：目的 MAC 用广播地址 `ff:ff:ff:ff:ff:ff`。因为收发都用 libpcap 在混杂模式下进行，不依赖内核按 MAC 收包，所以无需维护邻居 MAC 表，简化实现。
- **IP 首部**：标准 20 字节首部，**上层协议号选用保留值 253**（RFC 3692 为实验保留 253/254，内核没有对应的协议处理逻辑，不会干扰我们的分组）。IP 首部校验和必须正确计算（见下文“坑”）。
- **MTP（Mini Transport Protocol）首部**：极简设计，仅 4 个字段：

| 字段       | 长度 | 含义                                                    |
|----------|----|-------------------------------------------------------|
| conn_id  | 2B | <b>连接号</b> ：标识一条连接（环中一条边），也作为聚合位图的比特位                 |
| seq_num  | 4B | <b>序列号</b> ：按分组计数，从 0 开始                              |
| ack_flag | 1B | <b>ACK 标志</b> ：0=数据分组，1=ACK 分组                        |
| op       | 1B | <b>Operation</b> ：TRANSMISSION（普通传输）/ ALLREDUCE（触发聚合） |

- **载荷**：固定长度 `PAYLOAD_LEN = 1024` 字节。本实验假设用户传入的数据长度是 `PAYLOAD_LEN` 的整数倍，于是每个数据分组都恰好携带满载荷 `PAYLOAD_LEN` ——这样聚合器 slot 无需记录载荷长度，收发与聚合逻辑都更简单。（实际系统若数据不是整数倍，可对最后一个分组做填充对齐；本实验不处理这种情况、也不实现填充。）聚合时把载荷视为一组 32 位整数；本实验所有节点同架构，按主机字节序解释即可，无需 `htonl`。

注意：`conn_id` 取代了旧版本里用 (srcIP, dstIP, srcPort, dstPort) 四元组来识别流的做法。有了连接号，交换机一眼就能知道一个分组属于哪条连接、来自哪个 rank，分类逻辑大大简化。

## 收发架构：后台接收线程 + 接收缓冲区

每个容器内：

- **每个网络接口维护一个后台接收线程**（host 只有一个接口；router 每个端口一个线程），线程用 `pcap_next_ex` 持续抓“进入方向”的帧（`pcap_setdirection(PCAP_D_IN)`，避免抓到自己注入的帧造成环路）。
- 主机端：接收线程按 `conn_id` 把分组**分发**到对应连接的**接收缓冲区**（每条连接一个环形队列）。`m_send` / `m_recv` 只从自己连接的缓冲区里取分组，互不干扰。
- 路由器端：所有端口的接收线程把帧放入一个**共享的分组缓冲区**，主循环从中取帧处理。

发送统一通过 `pcap_inject` 注入本端口（主机端多线程发送时加锁）。

## 实验内容

实验分四个阶段循序渐进，对应四种运行模式（`route` / `transport` / `shift` / `allreduce`）。最终提交的 `lab.c` 是包含全部功能的完整实现。

## 阶段一：路由逻辑

**目标：**打通 libpcap 收发与路由转发，做单分组收发测试。

主机端实现三个函数（此阶段只做最简单的“收一个/发一个”，不含可靠性）：

- `init_conn(conn_id, local_ip, remote_ip)`：建立一条连接（登记连接号与两端 IP，分配接收缓冲区）；
- `m_send(conn, buf, size, op)`：把数据封装成 MTP 分组注入网卡；
- `m_recv(conn, buf, size, op)`：从连接缓冲区取出分组写入 `buf`。

路由器端实现 `Router()`：

1. 启动时按配置 `ranks.cfg` 建立**转发表**：目的主机 IP → 连接该主机的路由器端口；
2. 收到分组后解析 IP 首部，按目的 IP 查转发表，从对应端口注入（不匹配则丢弃）。

**测试（`route`）**：rank 0 把 `input-0.data`（256 个整数）发给 rank 3，rank 3 接收后写入 `output-3.data`，比对 `output-3.data` 与 `input-0.data` 是否逐字节相同。

一个常见的坑：如果测试不通过，请优先检查 **IP 首部校验和** 的计算是否正确。错误的校验和在某些链路/配置下会被丢弃。本模板用标准的 16 位反码求和实现 `ip_checksum()`，务必在每次改写 IP 首部后重算。另外注意 IP 地址的网络序/主机序转换。

## 阶段二：传输逻辑

**目标：**在 `m_send` / `m_recv` 中实现可靠传输。参考实验五，但**直接构建在我们自己的 MTP 首部之上**（不再像实验五那样建立在 UDP socket 之上），`op` 字段取 `TRANSMISSION`。

协议简化如下：

- **固定窗口**（`WINDOW = 32` 个分组），**不做拥塞控制、不做流量控制**；
- **超时重传**：固定 RTO（`RTO_US = 50ms`），不做 RTT 估计；
- **独立 ACK + 选择重传**：每个分组单独确认，接收端对每个收到的数据分组（含重复）都回 ACK；
- **初始序列号为 0**。

发送端 `m_send` 主循环：收 ACK → 滑动窗口后沿 → 发送窗口内未确认/已超时的分组 → 全部确认则结束。接收端 `m_recv` 主循环：收数据分组 → 写入对应偏移、回 ACK → 全部收齐后再等 0.5s（继续补发 ACK，以防最后的 ACK 丢失）→ 结束。

**测试（`transport`）**：rank 0 向 rank 3 可靠传输 1MB 数据。可用 `tc netem` 注入丢包后再测，验证重传逻辑。

## 阶段三：Shift 逻辑

**目标：**4 个 host 组成环，实现环形移位。

```
1 int shift(int group_id, char* src_addr, char* dst_addr, uint32_t size);
```

语义：每个 host 把自己 `src_addr` 中的数据传给**后继 rank** 的 `dst_addr`。等价地，从本机视角看：向后继发送 `src_addr`，同时从前驱接收到 `dst_addr`，最终 `dst_addr[j] = src_addr[(j-1+N)%N]`。

实现：每个 host 建立两条连接（都走传输层，`op = TRANSMISSION`）：

- **后继连接**：连接号 = 本机 rank，向后继发送数据（用一个线程跑 `m_send`）；
- **前驱连接**：连接号 = 前驱 rank，从前驱接收数据（主线程或另一线程跑 `m_recv`）。

路由器仍然只做转发（`Router()`），把数据从发送方转发给后继，把 ACK 转发回发送方。

**测试（`shift`）**：每个 host 从 `input-<rank>.data` 读取并发送，运行后把收到的内容写入 `output-<rank>.data`；按语义，`output-j.data` 应与前驱的输入 `input-<(j-1+N)%N>.data` 相同。

## 阶段四：AllReduce 逻辑

**目标**：4 个 host 组成环，实现在网 AllReduce。

代码块

```
1 int allreduce(int group_id, char* src_addr, char* dst_addr, uint32_t size);
```

**主机端逻辑与 Shift 完全相同**（向后继发、从前驱收），唯一区别是 `op = ALLREDUCE`。也就是说，主机像做 Shift 一样把数据发给后继，但路由器会把这些分组**拦截下来做聚合**，并在聚合完成后把结果广播回所有成员。运行后每个 host 的 `dst_addr` 内的数据都等于所有 rank 的 `src_addr` 内的数据之和。

路由器端实现 `INC()`（转发逻辑与 `Router()` 相似，但增加聚合逻辑）：

## 数据结构

- **连接上下文** `conn_ctx[k]`（按连接号/rank 索引）：连接 k 表示环上的边 `host[k] → host[k+1]`，记录 `src_ip = host[k]`、`dst_ip = host[k+1]`、`rank = k`；
- **聚合器数组** `agtr[]`，长度 `A = AGTR_ARRAY_SIZE`。**A 必须  $\geq 2 \cdot W$** （W 为窗口大小），本实验取最小值 `A = 2W = 64`。每个 slot 只需两样东西：
  - `bitmap`：已贡献的 rank 位图；
  - `payload[]`：按 int32 累加的载荷（长度固定为 `PAYLOAD_LEN`，因数据是整数倍、每个分组满载荷，故**不必再记录载荷长度**）。



- 由于消息长度（默认 1024 个分组）**远大于** A，同一个物理 slot 会先后服务多个序列号，因此采用**取模寻址**  $\text{slot} = \text{agtr}[\text{seq} \% A]$ ，并配合下面的“循环复用”机制。

## 转发/聚合逻辑（收到一个分组）

1. 解析 IP + MTP；若上层协议不是 MTP（253）则忽略；
2. 若是 **ACK 分组**或 **op 不是 ALLREDUCE 的数据**：按目的 IP 直接转发（复用 Router() 的转发表）；
3. 若是 **AllReduce 数据分组**：取连接号  $k$ （= 贡献者 rank）、序列号  $\text{seq}$ ，定位  $a = \text{agtr}[\text{seq} \% A]$ ， $\text{bit} = 1 \ll k$ ：
  - a. **叠加**：若 `bitmap` 中 rank  $k$  的比特**尚未置位**，则把整段载荷（ $\text{PAYLOAD\_LEN}/4$  个 int32）累加进 `a->payload`，并置位该比特（记录已叠加）。若已置位则跳过叠加（重复分组不重复累加）。
  - b. **判完成**：若 `bitmap` **未集齐**（ $\neq$  满位图  $(1 \ll N) - 1$ ） $\rightarrow$  丢弃该分组（什么都不做，等其余成员到齐）；若**已集齐** $\rightarrow$  清空“一个窗口外”的聚合器 `clear(seq + W)`，并**广播结果**（对每条连接  $k$  复制一份结果分组，改写首部  $\text{src}=\text{host}[k]$ 、 $\text{dst}=\text{host}[k+1]$ 、连接号= $k$ 、序列号= $\text{seq}$ ，从通往  $\text{host}[k+1]$  的端口发出）。

这套逻辑天然兼顾了重传：某成员没收到结果而**重传**该序列号时，分组到达时 `bitmap` 已是满位图，第 1 步不再重复叠加，第 2 步直接**重新广播**结果（幂等恢复）。无需额外的“已广播”标志位。

**广播 = 把结果写回所有连接**：聚合后的和分别以“连接  $k$  的数据”形态发给  $\text{host}[k+1]$ ，于是每个主机都从自己的前驱连接收到完整的和，并回 ACK。

## 聚合器复用（recycling）——与窗口协议联合设计

这是本实验的核心难点，参考 NetReduce / EPIC（Mode-II）。规则只有一条：

**每当一个序列号  $s$  聚合集齐，就清空“一个窗口外”的聚合器 `clear(s + W)`**——即把物理 slot  $(s+W) \% A$  的位图与载荷清零，为后续序列号  $s+W$  腾出一个干净的 slot。

为什么这样就正确，且不会发生“旧数据污染新聚合”或“新分组找不到干净 slot”？依赖**窗口不变式**：

- 发送端窗口为  $W$ ：要发送序列号  $s$ ，必须  $\text{base} > s - W$ ，即该连接**已确认到  $s - W$** 。在 AllReduce 中，“连接  $c$  确认了  $s - W$ ”意味着  $c$  的后继已收到  $s - W$  的结果——而结果只有在  $s - W$  **被所有成员贡献**、聚合完成后才广播。
  - 因此：**交换机只要看到任何一个序列号  $s$  的分组，就能断定  $s - W$  已被全体聚合完成。**
- 发送端窗口为  $W$ ：发送序列号  $s$  意味着  $s$  在窗口内，“ $s + W$  在窗口外部”。
  - 因此：**交换机只要看到序列号  $s$  “聚合”完成，就能断定  $s + W$  在“全体”之外，可安全清空。**

- 取  $A = 2W$ ：序列号  $s$  完成时清空  $(s+W) \% A$ ，它恰好是序列号  $s+W$  将要使用的 slot ——  $s+W$  正是  $s$  完成（窗口前移一格）后新获准发送的序列号。于是**新序列号到来前，它的 slot 一定已被清零**。
- 初始的  $0..A-1$  代由 `calloc` 清零，不必额外处理。
- 为什么聚合器不需要记录序列号来识别迟到的旧分组？** 清空  $(s+W) \% A$  时，该物理 slot 的上一位占用者是序列号  $s-W$ （与  $s+W$  相差  $A=2W$ ）；而  $s$  完成  $\Rightarrow$  全体已确认到  $s-W \Rightarrow s-W$  早已滑出所有发送方的窗口，**不会再被发送**。再加上一个工程假设——**网络中没有延迟极久的陈旧分组**——就不会出现“旧序列号的分组落进已被回收的 slot”的情况，因此 slot 里**无需保存序列号标签**去甄别，逻辑得以进一步简化。

## 为什么整体可靠

- 路由器在 slot 聚合完成前**不转发**这些数据分组（相当于“吞掉”）。发送方收不到结果就不前进，而结果只有在**所有** rank 到齐后才广播；位图保证重传不会被重复累加。
- 任一数据分组丢失  $\rightarrow$  对应 slot 不满  $\rightarrow$  不广播  $\rightarrow$  发送方超时重传，直至集齐。
- 结果分组 / ACK 丢失  $\rightarrow$  发送方重传  $\rightarrow$  命中已满的 slot  $\rightarrow$  **重新广播**（幂等恢复）。
- 因为用的是**独立 ACK / 选择重传**（而非 TCP 累积确认），接收端允许乱序，所以路由器**一旦某 slot 完成即可立即广播**，无需维护“按序广播游标”，逻辑比 TCP 版本简单得多。

## 开发与测试

代码块

```
1  make setup_env           # 安装 libpcap-dev、配置 docker 镜像源、构建 node 镜像（只需一次）
2  make setup_topo          # 启动星形拓扑（router + host1..4）
3  make                     # 编译，生成 inc
4
5  make test_route          # 阶段一：单分组路由收发
6  make test_transport      # 阶段二：1MB 可靠传输
7  make test_shift          # 阶段三：环形移位
8  make test_allreduce      # 阶段四：在网 AllReduce
9  make test                # 依次跑完四个阶段
10
11 make clean_topo          # 销毁拓扑
```

## 数据驱动测试方法

测试**不预设数据规律**，而是“生成输入文件  $\rightarrow$  运行  $\rightarrow$  比对输出文件”，全程以文件为准：

测试用一个小脚本 `helper.sh`（用 `seq` / `awk` / `paste` / `cmp` 等命令实现）完成三件事：



1. `helper.sh gen N nints` 为每个 rank 生成发送内容文件 `input-<rank>.data`（每行一个 int32，内容随 rank 与下标变化）；
2. 运行后，每个收数据的主机把结果写入 `output-<rank>.data`；
3. `helper.sh check mode N` 按各原语的语义**比对文件**（`cmp` 逐字节，不做任何解析式假设）：
  - **transport**: `output-<N-1>.data` 应与发送端的 `input-0.data` **逐字节相同**；
  - **shift**: `output-j.data` 应与**前驱**的输入 `input-<(j-1+N)%N>.data` 相同（每个主机收到的正是前驱发出的内容）；
  - **allreduce**: 先由独立的单机参考 `helper.sh sum N` 把所有输入按元素求和写入 `expected-allreduce.data`，再要求**每个**rank 的 `output-<rank>.data` 与之相比。`make test_*` 会自动完成“生成输入 →（allreduce 额外生成单机求和参考）→ 运行 → 文件比对”。

正确的输出示例（AllReduce, N=4）：

代码块

```
1  ===== check (allreduce) =====
2  rank 0 allreduce (output-0.data == expected-allreduce.data): PASS
3  rank 1 allreduce (output-1.data == expected-allreduce.data): PASS
4  rank 2 allreduce (output-2.data == expected-allreduce.data): PASS
5  rank 3 allreduce (output-3.data == expected-allreduce.data): PASS
6  ==> RESULT: PASS
```

## 提交文件

通过教学网提交 `lab.c`。

模板按上述参考方案构造并已验证可行（四个阶段在 docker 环境下均通过测试，且在 5%~10% 丢包注入下 仍能正确完成）。学生可以自行修改 `lab.c` 的内容，只要其他文件不变、测试通过即可。

## 其他（调试建议）

1. `printf` 调试后记得 `fflush(stdout)`，否则进程被强杀时可能丢失输出（回忆实验五）。
2. 另开一个终端用 `tcpdump -i [接口] -ne -X` 观察分组是否按预期到达、载荷长度是否正确。注意我们的协议号是 253，可用 `tcpdump 'ip proto 253'` 过滤。
3. 注入丢包测试重传：`docker exec host1 tc qdisc add dev host1-eth0 root netem loss 10%`，取消用 `tc qdisc del dev host1-eth0 root`。
4. 用 `make kill` 清理残留进程；用 `make clean_topo && make setup_topo` 重建拓扑。

5. 离线/无 docker 自测：仓库提供 `netns_test.sh`，用非特权用户命名空间搭建同样的拓扑，无需 `sudo` 或 `docker` 即可端到端验证：`unshare -rUmn bash netns_test.sh allreduce`（可选第二个参数为丢包率，如 `... allreduce 5`）。

## 思考题

1. 为什么 TCP socket 无法满足在网计算对分组边界的要求？我们用了哪些手段把分组边界“夺”回来？
2. 路由器在 slot 聚合完成前丢弃数据分组，这是否会破坏可靠性？为什么不会？
3. 为什么用了独立 ACK / 选择重传后，路由器可以乱序广播、不需要按序游标？
4. 聚合器复用为什么要求数组长度  $A \geq 2W$ ？如果取  $A = W$  会出什么问题？
5. 为什么“清空一个窗口外的 slot”能保证新序列号到来时其 slot 一定是干净的？（提示：用发送窗口不变式论证  $s$  完成  $\Rightarrow$  全体已确认  $s-W$ ）
6. 我们删掉了 slot 里的“序列号标签”。这依赖了什么假设？在什么情况下（例如某个分组在网络里延迟极久后才到达）这个简化可能导致聚合结果出错？真实系统可以如何防住这种情况？